

The 8085 Master Manual

Volume IV: Expanding the Universe
(Address Decoding & Interfacing)

A Grassroots Engineering Guide

*Constructing the bridge between the Microprocessor and the Physical
World.*

*"An interface is a boundary across which two independent entities meet
and communicate."*

Chapter 1: The Philosophy of Interfacing

1.1 The Isolation of the Core

Up to this point, we have meticulously constructed the internal architecture of the 8085 microprocessor. We have established its registers, its arithmetic logic unit, and its rigorous timing sequencers. However, a microprocessor in isolation is entirely useless. It is a brain in a jar. It possesses immense processing capability but has no memory to store programs permanently and no means to interact with human operators or physical sensors.

To build a functional computer, we must connect the 8085 to external components:

- **Memory Devices:** Read-Only Memory (ROM) to store permanent boot instructions, and Random Access Memory (RAM) to store temporary variables and user programs.
- **Peripheral Devices (I/O):** Keyboards, displays, analog-to-digital converters (ADC), and sensors.

1.2 Defining the Interface

The act of connecting these external entities to the microprocessor is called **Interfacing**.

THE DEFINITION OF INTERFACING

Interfacing is the concept and physical implementation of an interaction point between components. In the 8085, it is the hardware circuitry (and associated software protocols) that matches the stringent timing, electrical, and logical signal requirements of the external memory and I/O devices with the signals provided by the microprocessor's System Bus.

Chapter 2: The Anatomy of a Memory Chip

Before we can interface memory to the microprocessor, we must intimately understand what a memory chip expects from the outside world.

2.1 Signal Requirements of Memory

Whether a chip is RAM or EPROM, it physically consists of an immense grid of microscopic storage registers. To successfully read from or write to a specific register, the memory chip demands three distinct types of electrical information, perfectly mirroring the CPU's System Bus:

1. **Address Lines** ($A_0 - A_n$): To select a specific internal register.
2. **Data Lines** ($D_0 - D_7$): The physical pathway for the 8-bit data to enter or leave the chip.
3. **Control Lines**: Signals that enable the chip and dictate the direction of data flow.

2.2 The Mathematics of Capacity

The storage capacity of a memory chip dictates exactly how many Address Lines it must possess. The relationship is governed by the formula: $C = 2^N$, where C is the number of internal registers and N is the number of address pins.

Example 1: A 2048-Byte (2 KB) RAM Chip

To uniquely identify 2048 distinct registers, the chip requires $\log_2(2048) = 11$ address lines. These are labelled A_0 through A_{10} .

Example 2: A 4096-Byte (4 KB) EPROM Chip

To identify 4096 registers, the chip requires $\log_2(4096) = 12$ address lines. These are labelled A_0 through A_{11} .

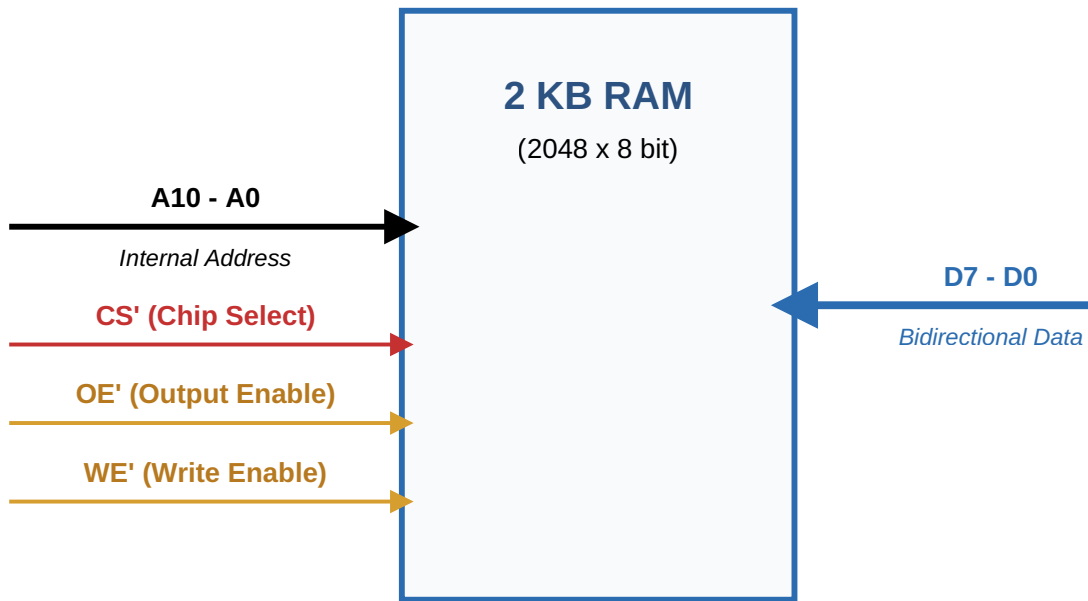
2.3 The Control Pins (CS', OE', WE')

A memory chip does not permanently broadcast data onto the data bus. If it did, it would instantly cause an electrical short-circuit the moment another chip attempted to broadcast. Memory chips use internal *tri-state buffers* to disconnect themselves from the data bus until explicitly commanded to connect.

- **Chip Select (CS') or Chip Enable (CE'):** This is the master switch of the memory chip. It is an active-low pin. If $CS' = 1$, the entire chip is asleep. If $CS' = 0$, the chip wakes up.

- **Output Enable (OE'):** Active-low. When $CS' = 0$ and $OE' = 0$, the chip pushes the data from the selected register out onto the Data Bus (A Read Operation).
- **Write Enable (WE'):** Active-low. Found only on RAM chips. When $CS' = 0$ and $WE' = 0$, the chip absorbs the data currently sitting on the Data Bus (A Write Operation).

2.4 Visualizing the Generic Memory Interfacing Block



The CPU outputs a 16-bit address ($A_0 - A_{15}$). However, our 2 KB RAM chip only has 11 address pins ($A_0 - A_{10}$). We wire the CPU's lower 11 address lines directly to the memory chip's 11 address pins.

The Central Question of Interfacing: What do we do with the remaining 5 address lines from the CPU ($A_{11}, A_{12}, A_{13}, A_{14}, A_{15}$)?

We cannot ignore them. To place this RAM chip at one specific, unique 64KB location, we must use the remaining 5 lines to control the RAM's **Chip Select** (CS') pin. This process is called **Address Decoding**.

Chapter 3: The Imperative of Address Decoding

EXAMPLE: DECODING A 2 KB RAM TO BASE ADDRESS 2000H

We have a 2 KB RAM (Requires $A_0 - A_{10}$).

Remaining lines for decoding: A_{15} , A_{14} , A_{13} , A_{12} , A_{11} .

We want the chip to activate when the address is in the 2000H range.

Let us examine 2000H in binary:

A15	A14	A13	A12	A11	A10 - A0
0	0	1	0	0	(Internal chip address)

For the CS' pin to receive a 0, the NAND gate must output a 0. A NAND gate outputs 0 only when all inputs are 1. Therefore, we must invert the zeros before feeding them into the NAND gate.

LOGIC EQUATION:

$$CS' = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{13} \cdot \overline{A_{12}} \cdot \overline{A_{11}}$$

Chapter 4: Absolute vs. Partial Decoding

The mathematical rigor with which an engineer applies address decoding directly affects the physical layout of the memory map.

4.1 Absolute Decoding (Exhaustive Decoding)

In **Absolute Decoding**, every single one of the 16 address lines is explicitly accounted for. Because no address lines are left un-decoded, the memory chip responds to one, and *only one*, specific range of addresses in the 64 KB space. There is no ambiguity.

4.2 Partial Decoding and The Phenomenon of Image Addresses

In smaller systems, engineers may choose to leave one or more of the higher-order address lines completely unconnected. This is **Partial Decoding**.

SCENARIO: DERIVING FOLDBACK ADDRESSES

We interface a 256-Byte RAM chip (Requires $A_0 - A_7$).

We design a partial decoder that only checks three lines: $A_{15}=0$, $A_{14}=0$, $A_{13}=1$.

The remaining lines (A_{12} , A_{11} , A_{10} , A_9 , A_8) are left unconnected (Don't Cares).

A15	A14	A13	A12	A11	A10	A9	A8	A7...A0
0	0	1	X	X	X	X	X	Internal

CALCULATING THE BASE ADDRESS:

The Base Address is defined as the range when all "Don't Care" bits are forced to 0.

Start: 0010 0000 0000 0000 = 2000H

End: 0010 0000 1111 1111 = 20FFH

CALCULATING IMAGE ADDRESS 1:

What if the CPU accidentally flips A_8 to 1? The decoder only looks at A_{15} - A_{13} , so it still activates the chip.

Start: 0010 0001 0000 0000 = 2100H

End: 0010 0001 1111 1111 = 21FFH

CALCULATING TOTAL IMAGE ADDRESSES:

There are 5 unconnected address lines.

Total duplicate mappings = $2^5 = 32$.

This single 256-Byte chip will occupy 32 different spaces (consuming 8 KB of total address space) inside the 8085's memory map.